

Ledger Architecture Review — Technical ADR

Status: Proposed

Scope: Financial ledger implementation

Primary concerns: Correctness, scalability, lifecycle management, operational safety

1. Append-only at scale

The implementation uses an immutable, append-only ledger as the system of record. This provides deterministic history, auditability and the ability to reconstruct financial state.

The design intentionally separates **financial history** from **derived operational state**. This is important because scaling the read path should not require compromising the immutability of the underlying ledger.

The Ledger is calculate each time new event reach the system

Debit / Credit/ Authorization(heldAmount)/reversal(Only Debit/Credit event)/settlement

2. Decision: Append-Only Ledger at Scale

What breaks first at 100×?

At **100× the current volume**, the first pressure point is the in-memory replay state: the engine retains accounts, balances, active holds, authorization state, and other derived information while processing the event history. The event history itself also grows without bound because every business change remains part of the ledger.

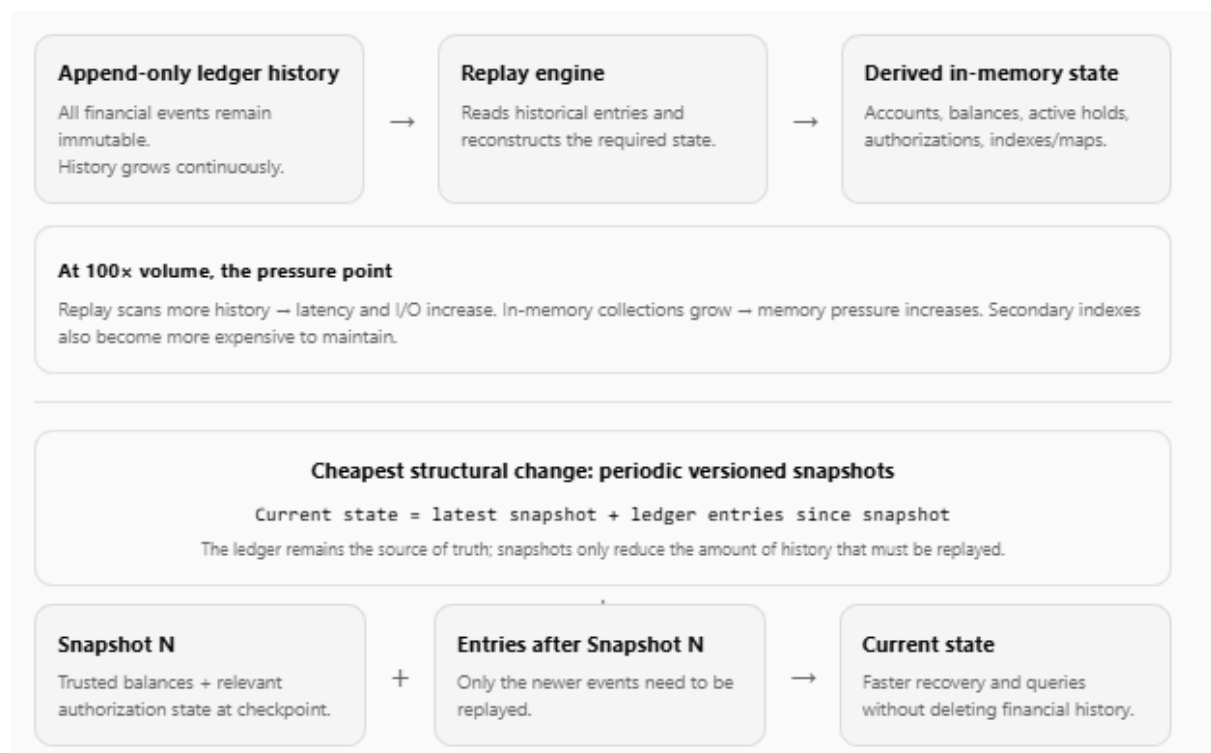
The design accumulates unbounded state in the **ledger history itself**, along with its indexes and any derived state maintained from it. Financial history cannot simply be deleted, so this growth is inherent to the model.

If a balance or authorization query requires scanning/replaying an increasingly large transaction history, latency and I/O grow with historical volume. Secondary indexes also become increasingly expensive to maintain and the list and used map will be memory consuming.

Current state = latest snapshot + ledger entries since that snapshot

The cheapest structural change would be to introduce **periodic, versioned account snapshots/checkpoints**. Replay would start from the latest trusted snapshot and process only subsequent events. This reduces replay time without abandoning the append-only source of truth.

For production, the event store would also need durable partitioning, retention/archival policy, deterministic replay, checksums or equivalent integrity controls, and operational monitoring. The CBUAE places importance on reliable records and audit trails for financial activity



Decision

Retain the append-only ledger as the authoritative source of truth.

At higher volume, introduce:

- time/account-based partitioning;
- indexed access paths for active accounts;

- periodic balance snapshots/checkpoints state;
- archival of immutable historical partitions(to be kept for 5 as per UAE-licensed bank).

The unbounded state is:

Ledger history → indexes → derived/snapshotpoint state

The ledger itself must remain unbounded from a business perspective because financial history cannot simply be deleted when it becomes inconvenient.

Cheapest structural change

Introduce **periodic snapshots**.

Instead of:

Replay transaction 1 ... transaction N

a balance query becomes:

Latest snapshot + transactions after snapshot

This reduces the amount of historical data required for normal reads without changing the source-of-truth model.

3. Decision: Value-Dated Entries

Technical distinction

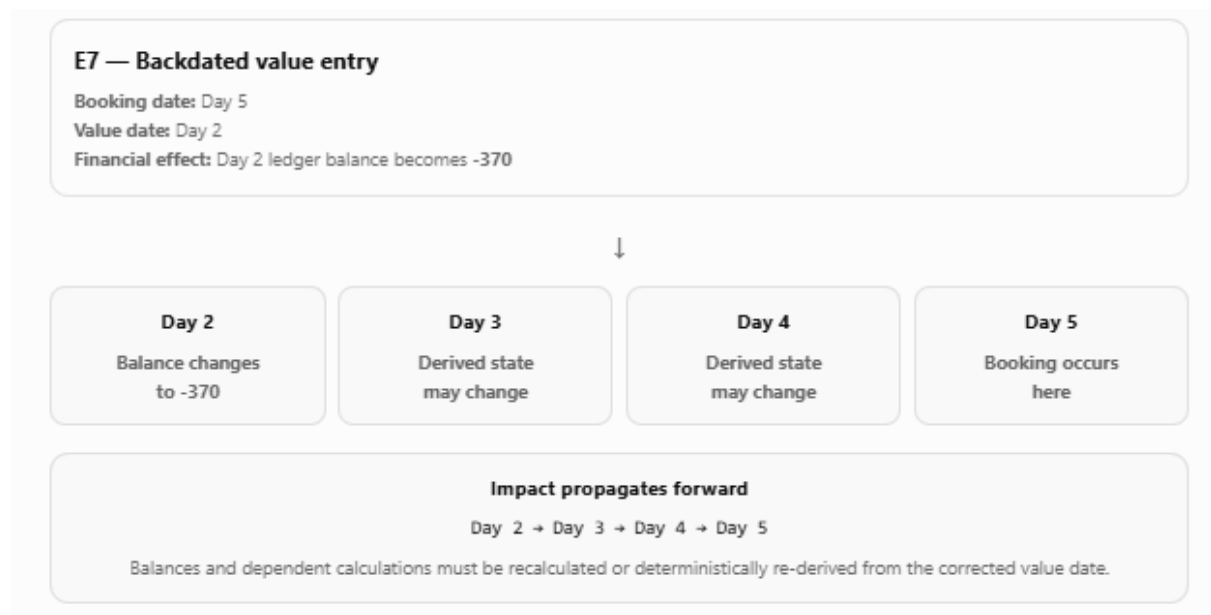
The implementation must distinguish:

- **event/booking timestamp** — when the system records the operation;
- **value date** — when the financial effect becomes effective;
- **settlement timestamp** — when the financial obligation is actually settled.

Keeping these concepts separate is important because a transaction can be booked on one day while affecting the ledger position of another day.

The main production consequence is **backdating**. If an event is booked on Day 5 but has a Day 2 value date, it changes the financial state from Day 2 onward. Therefore, derived balances and calculations for subsequent days may become stale and must be recalculated or otherwise deterministically re-derived.

For example:



This can affect overdraft fees, interest/accrual calculations, available balances, reconciliation, end-of-day processing, reporting and potentially an authorization that was created or settled during the affected period.

Production impact

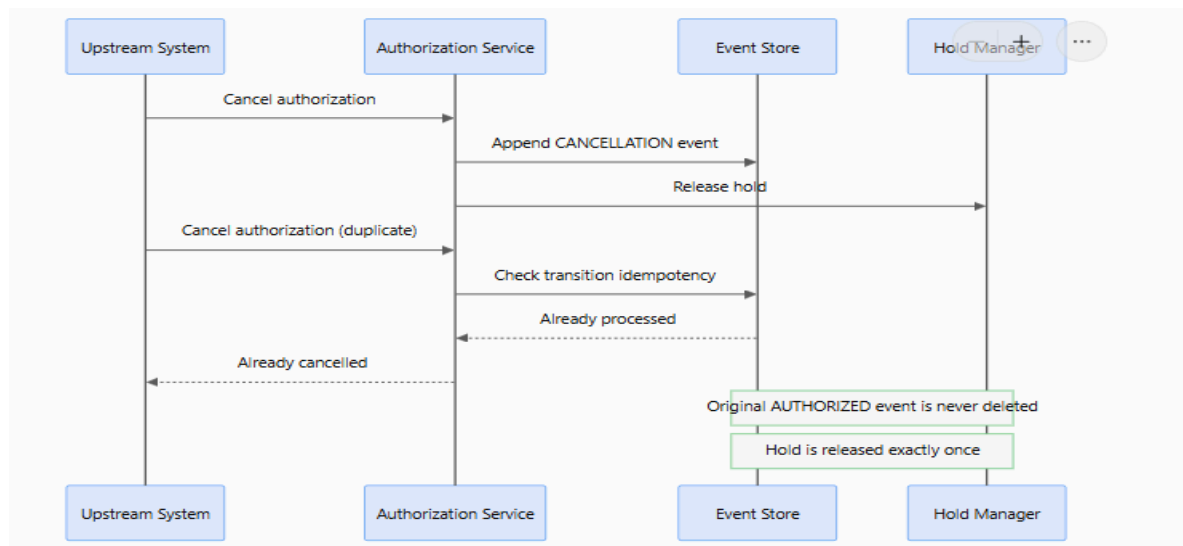
Value-dated entries therefore create an operational and audit surface around:

- historical balance reconstruction;
- reconciliation(Authorization| settlement event);
- interest and fee recalculation;
- end-of-day and cut-off processing;
- customer statements;
- downstream accounting/reporting;
- authorization and settlement state;
- investigation and audit.

The main production risk is unauthorized or incorrect backdating: a technically valid event can alter a prior financial position and consequently affect subsequent derived state.

4. Decision: Authorization State Machine

An authorization does not necessarily end through settlement. It may expire, be cancelled, reversed, declined, or become non-settleable for another reason(**Timeout / Exception, Account invalidation, Supersession**).



4.1 Key Invariants

- **Original authorization is never deleted or overwritten.**
- **Terminal authorizations cannot be settled.**
- **A hold is released exactly once.**
- **Every transition is auditable** with reason, timestamp, and actor/service where applicable.
- **Duplicate transitions are idempotent** and must not cause duplicate releases or settlements.
- **Authorization, hold, and ledger states remain distinct.**

4.2 Architectural Decision

Authorization termination is represented as a **new state transition/event**, not as mutation or deletion of the original authorization.

This preserves the audit trail and prevents an expired, cancelled, or reversed authorization from becoming accidentally settleable.

The production design therefore requires:

- idempotency keys;
- atomic state transitions;
- optimistic/pessimistic concurrency control where appropriate;
- unique constraints for externally supplied transaction identifiers;
- deterministic handling of duplicate requests.

The ledger remains append-only, but **state-transition validation must be atomic**.

5. What Was Deliberately Cut

The implementation intentionally remains an **in-memory replay exercise**. Production infrastructure and operational concerns were simplified to keep the solution focused on ledger correctness.

Simplification	Reason	Deferred production concern
Single logical storage model	Keep accounting semantics clear.	Sharding and horizontal scaling.
Simplified authorization lifecycle	Keep the core domain manageable.	Timeouts, retries, reversals and operational exceptions.
Simplified value dating	Avoid banking-calendar complexity.	Cut-offs, holidays and controlled backdating.
No reconciliation engine	Outside the core ledger responsibility.	Cross-system financial discrepancies.
No multi-region active/active	Avoid premature distributed complexity.	Regional failure and availability.
Basic concurrency model	Focus on ledger semantics.	High-contention accounts and distributed concurrency.

These are **scope decisions, not assumptions that the underlying production concerns do not exist.**

6. Production Architecture Evolution

The intended evolution is:

Phase 1 — Correctness

Immutable ledger + deterministic transitions + audit trail.

↓

Phase 2 — Operational scale

Partitioning + indexes + snapshots + idempotency + reconciliation.

↓

Phase 3 — Enterprise resilience

Archival/retention + multi-region recovery + operational controls + regulatory integrations.

This sequencing avoids introducing distributed-system complexity before it solves an actual bottleneck.

7. Production Gate

I would not consider the implementation production-ready until the following are demonstrated:

- atomic/idempotent financial operations;
- complete authorization terminal-state handling;
- controlled value-date modification;
- reconciliation with external systems;
- partitioning/snapshot strategy;
- recovery from partial failures and retries;
- access control and segregation of duties;
- monitoring for invariant violations and reconciliation breaks.

Architecture conclusion: The current implementation is a suitable correctness-focused ledger core. Production hardening should extend around it rather than weaken the append-only source-of-truth model.